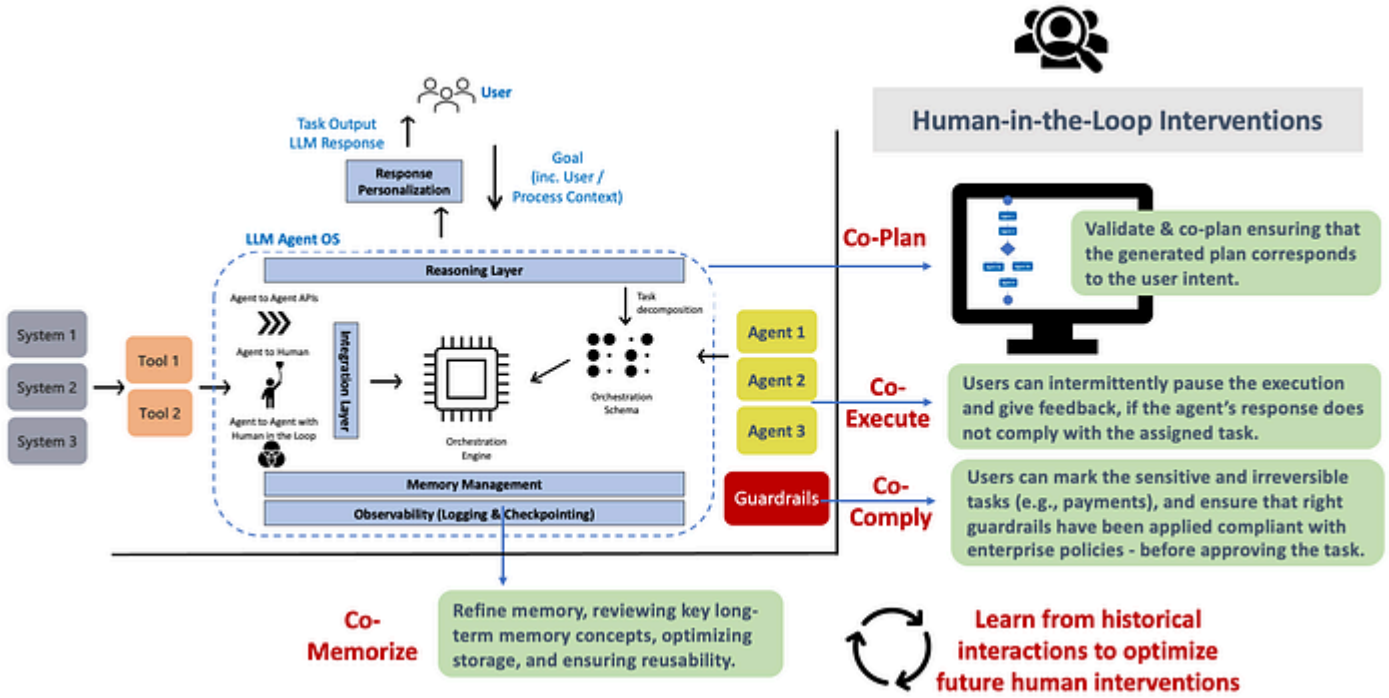


Human-in-the-Loop Strategy for Agentic AI

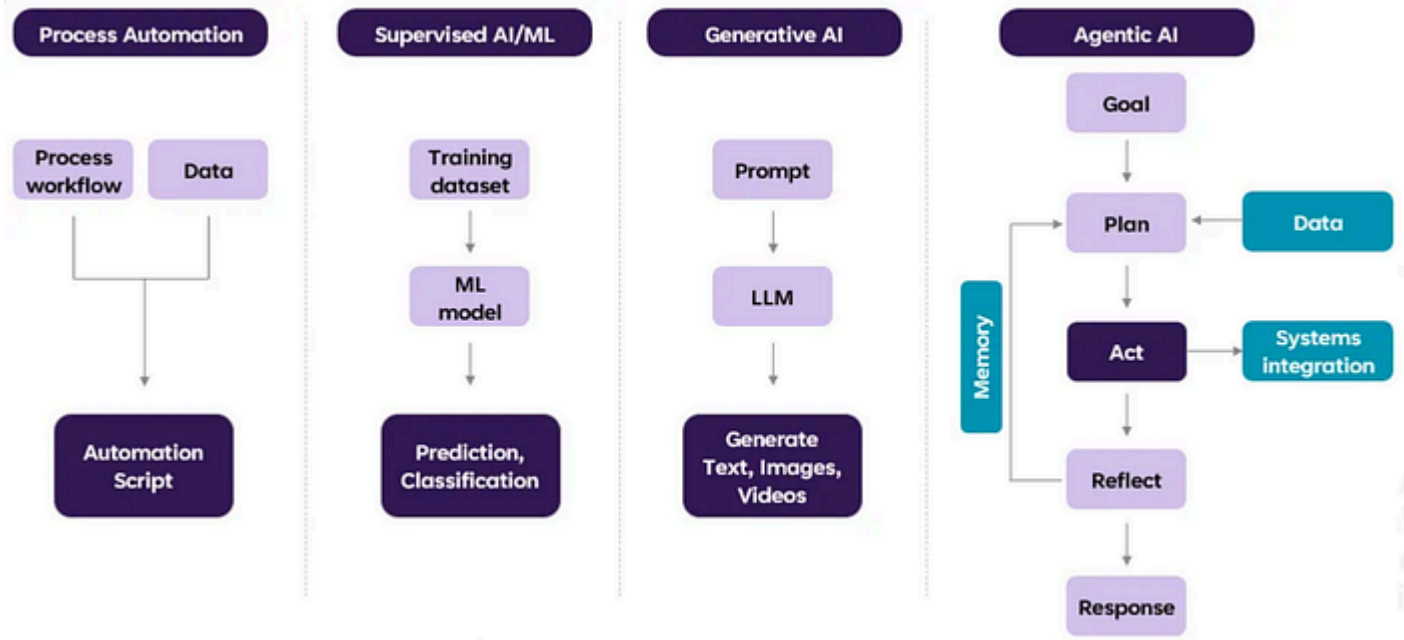
Collaborative Human-assisted Execution Model for AI Agents

Debmalya Biswas

AI Advances · ~10 min read · August 15, 2025 (Updated: August 15, 2025) · Free: No

1. Introduction to Agentic AI

The discussion around ChatGPT (in general, generative AI), has now evolved into agentic AI. While ChatGPT is primarily a chatbot that can generate text responses, AI agents can execute complex tasks autonomously, e.g., make a sale, plan a trip, make a flight booking, book a contractor to do a house job, order a pizza. The figure below illustrates the evolution of agentic AI systems. Fig. 1 below illustrates the evolution of agentic AI systems.



Bill Gates recently envisioned a future where we would have an AI agent that is able to process and respond to natural language and accomplish a number of different tasks. Gates used planning a trip as an example. Ordinarily, this would involve booking your hotel, flights, restaurants, etc. on your own. But an AI agent would be able to use its knowledge of your preferences to book and purchase those things on your behalf.

In short, the reason AI agents are so popular is because they can in principle be applied to any enterprise process that is executed manually today. We can basically agentify everything from a customer service desk, to industrial processes, e.g., HVAC optimization; to even leveraging agents to build the underlying software, data, and ML engineering pipelines.

> While the benefits of agentic AI systems are evident, they are also complex systems that are difficult to execute in a reliable and autonomous manner.

Given the unreliability of current LLMs / reasoning models, it is difficult to build the necessary **guardrails** needed to ensure that autonomous agents run in a reliable manner satisfying the applicable regulatory policies and controls.

much better in terms of task fulfillment. They also play a key role in providing the human oversight needed for autonomous agents today — to satisfy the legal and compliance requirements in any large enterprise :)

> Towards this end, we propose to integrate humans as first-class citizens in the agentic **lifecycle** (and not only as a supervisor / reviewer) with the appropriate UI/UX to support such interactions / interventions.

2. Agentic AI Lifecycle Management

The typical stages involved in building and operating AI agents are illustrated in Fig. 2.

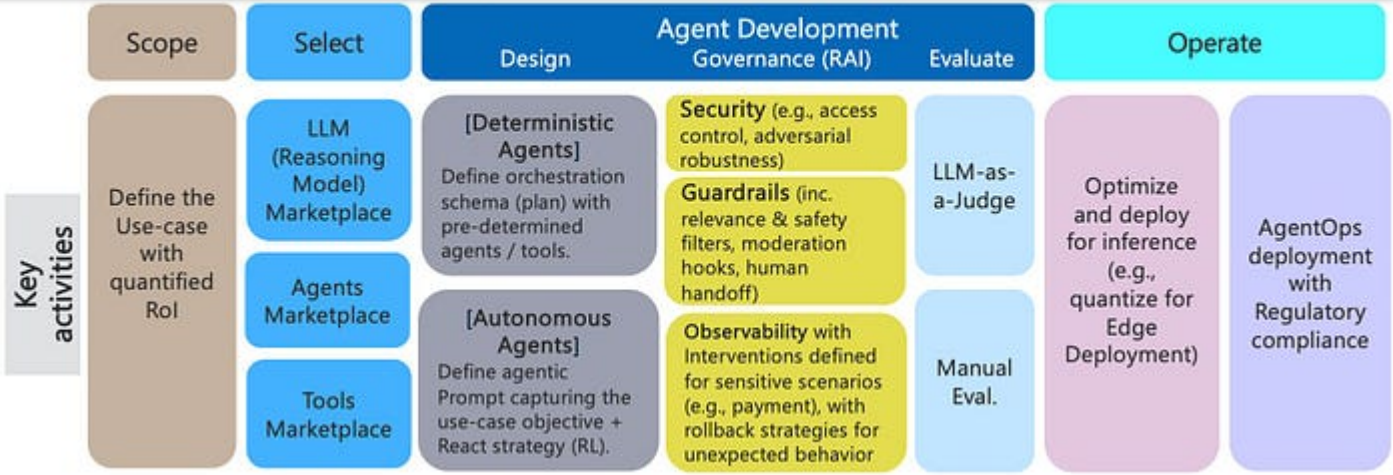


Fig. 2: Agentic AI Lifecycle Management (Image by Author)

First, we need to define the **use-case**: This includes defining the problem statement, understanding its business context, data requirements and availability, and setting clear objectives for the agentic AI solution quantifying return-on-investment (RoI).

Second, we need a **marketplace** of reasoning models / large language models (LLMs), agents and tools — defining agents and building enterprise tool integrations on the fly do not really work in practice :).

For example, the Agent2Agent (**A2A**) protocol specifies the notion of an Agent Card (a JSON document) that serves as a digital "business card" for agents. It

```
Copy

Identity: name, description, provider information.
Service Endpoint: The url where the A2A service can be reached.
A2A Capabilities: Supported protocol features like streaming or pushNotificatic
Authentication: Required authentication schemes (e.g., "Bearer", "OAuth2") to i
Skills: A list of specific tasks or functions the agent can perform (AgentSkil
```

Client agents can then discover remote agents by parsing their respective Agent Cards — to determine if a remote agent is suitable for a given task, how to structure requests for its skills, and how to communicate with it securely.

Along the same lines, the Model Context Protocol (**MCP**) specifies a similar mechanism for dynamic tool discovery. Using `mcp://` URIs, agents can resolve and retrieve comprehensive metadata information about tool capabilities, requirements, and interaction methods.

Both A2A & MCP are based on textual / natural language descriptions of agents and tools. In a previous paper, I have highlighted scenarios where this might not be sufficient and

> enable precise and automated discovery of tools and agents.

Third, we need to design the agentic **logic** (plan to fulfill the goal). Here, we need to differentiate between deterministic and autonomous agents — as their design and execution are very different.

In the case of **deterministic** agents, it primarily entails (statically) defining an orchestration schema at the start with pre-determined agents / tools. In contrast, in the case of **autonomous** agents, this is limited to specifying the use-case goal as a prompt to an LLM / reasoning model.

> The planner then dynamically defines the execution plan with the ability to adapt the plan in between — basically reacting to the environment based on observations in memory.

Fourth, we need to consider optimizing the deployment of agents for **inferencing**. With generative AI and the large size of LLMs, there was significant focus on optimizing / quantizing LLMs to small language models (SLMs). Given the enterprise workflow focus of most agents today, this seems to have been deprioritized a bit.

> focus as soon as we have more agents in production.

So this stage consists of proactively thinking about optimizing agentic deployments — to the extent that they can be deployed on edge devices. For more details, refer to my previous article on Agentic AI Inference Sizing.

Finally, we discuss the **governance** layer. Let's face it, without this layer, no agent is going into production in any enterprise — nor should they actually be allowed to do so. Case in point is the widely circulating letter by JP Morgan's CISO on the need for secure and resilient agentic architectures. Guardrails also seem to have become a first-class citizen in the agentic AI ecosystem with publication of OpenAI's Agent SDK.

> Overall, end-to-end observability is critical not only to recover from scenarios where the agent gets stuck, but also rollback strategies for scenarios where the agent starts going off-script.

In short, the key takeaway is building reliable and trusted agents in production entails more than writing a few lines of code -:)

3. Agentic AI Reference Architecture

Fig. 3 illustrates the key components of an agentic AI platform to accommodate the lifecycle stages identified in the previous section:

- Agents (and tools) marketplace
- Planner — reasoning layer
- Personalization layer
- Orchestration layer
- Observability layer (with logging, checkpointing, etc.)
- Integration layer (integrating with enterprise systems)
- Shared memory layer (long and short term memory)

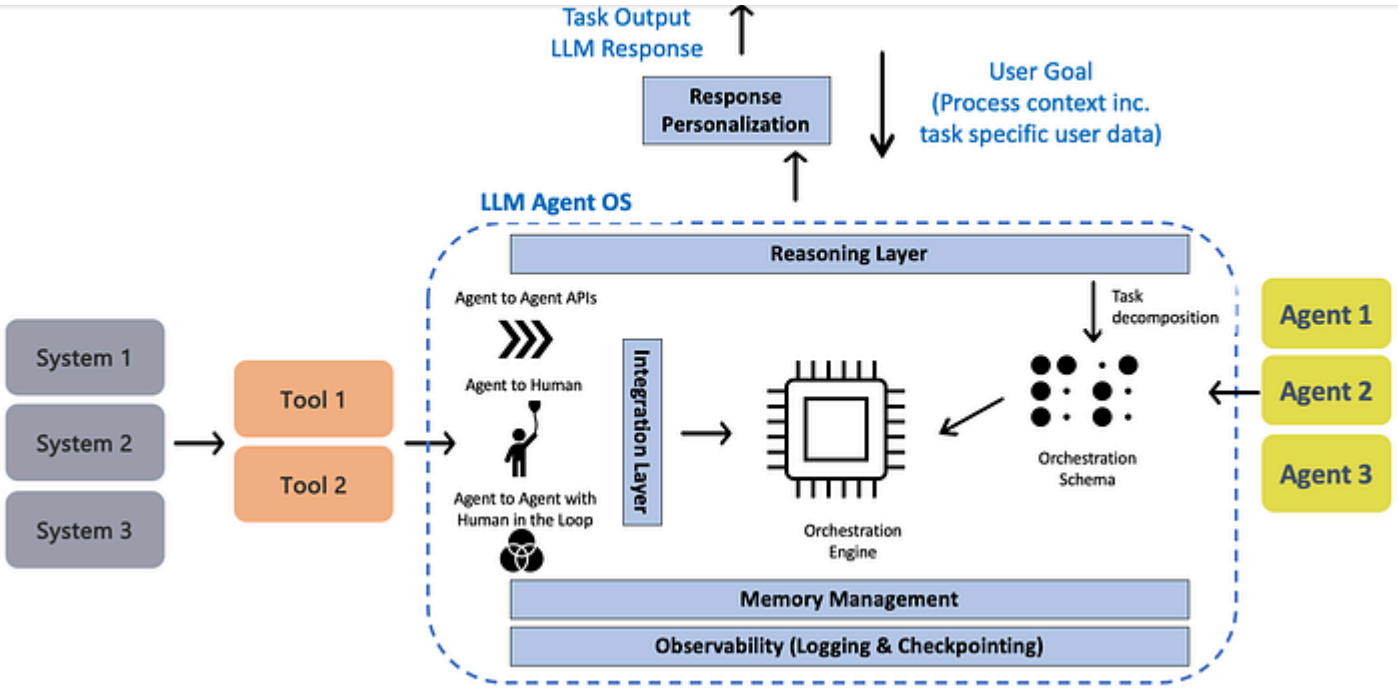


Fig. 3: Agentic AI Platform Reference Architecture (Image by Author)

Given a user task, we prompt a LLM for the task decomposition — this is the overlap with generative AI. Unfortunately, this also means that agentic AI systems today are limited by the **reasoning** capabilities of large language models (LLMs). For ex., the GPT4 task decomposition of the prompt

> Generate a tailored email campaign to achieve sales of USD 1 Million in 1 month, The applicable products and their performance metrics are available

is detailed in Fig. 4: (Analyze products) — (Identify target audience) — (Create tailored email campaign).

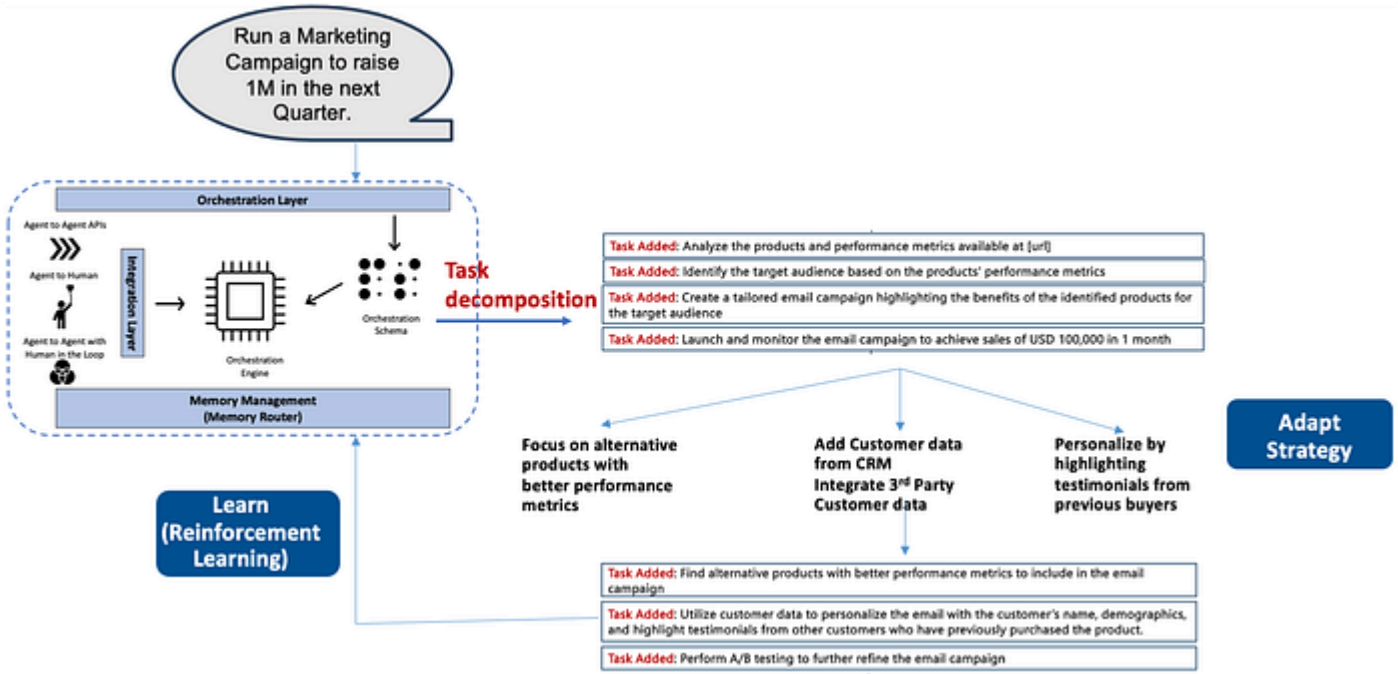


Fig. 4: Agentic AI execution of a Marketing Campaign (Image by Author)

The LLM then monitors the execution / environment and adapts autonomously as needed. In this case, the agent realised that it is not going to achieve its sales

This leads to the need for a **personalization** layer. Analogous to fine-tuning of LLMs to domain specific LLMs / SLMs,

> we argue that customization / fine-tuning of (generic) AI agents will be needed with respect to enterprise specific context (of applicable user personas and use-cases) to drive their enterprise adoption.

Fig. 5 illustrates the reference architecture to perform user persona based fine-tuning of AI agents. Refer to my previous article on Personalizing UX for Agentic AI for a detailed discussion on this topic.

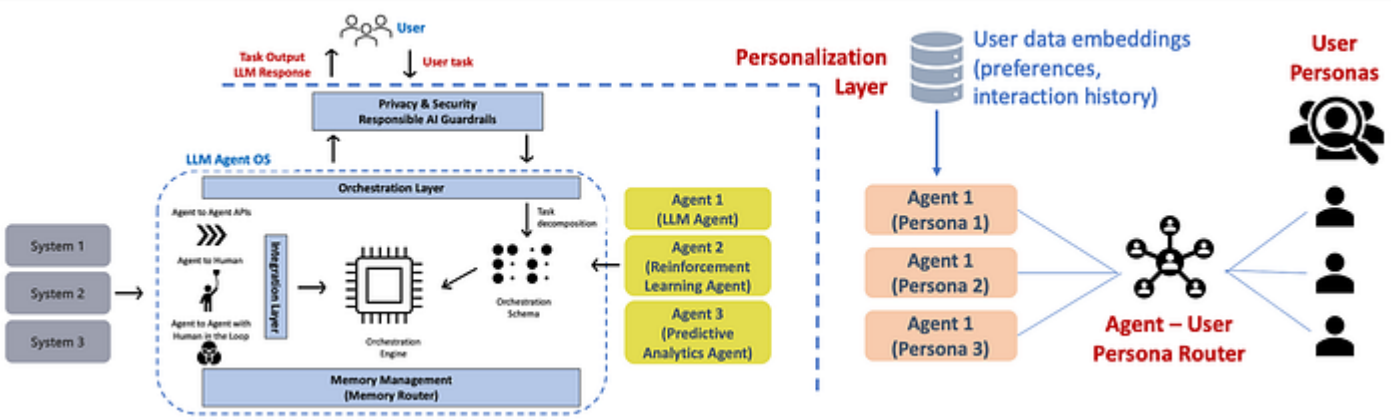


Fig. 5: User persona based Fine-tuning of AI agents (Image by Author)

agent API providing output for human consumption, human triggering an AI agent, AI agent-to-agent with human in the Loop. The integration patterns need to be supported by the underlying AgentOps platform.

It is also important to mention that integration with enterprise systems (e.g., CRM in this case) will be needed for most use-cases. This is for instance enabled by MCP by (dynamically) connecting tools to external systems where enterprise data resides.

Given the long-running nature of such complex tasks, **memory management** is key for Agentic AI systems. Once the initial email campaign is launched, the agent needs to monitor the campaign for 1-month.

> This entails both context sharing between tasks and maintaining execution context over long periods.

The standard approach here is to save the embedding representation of agent information into a vector store database that can support maximum inner product search (MIPS). For fast retrieval, the approximate nearest neighbors

Fig. 6 illustrates comprehensive memory management for agentic AI systems, including both short-term and long-term memory modules. Refer to my previous article on Long-term Memory for Agentic AI for a detailed discussion on this topic.

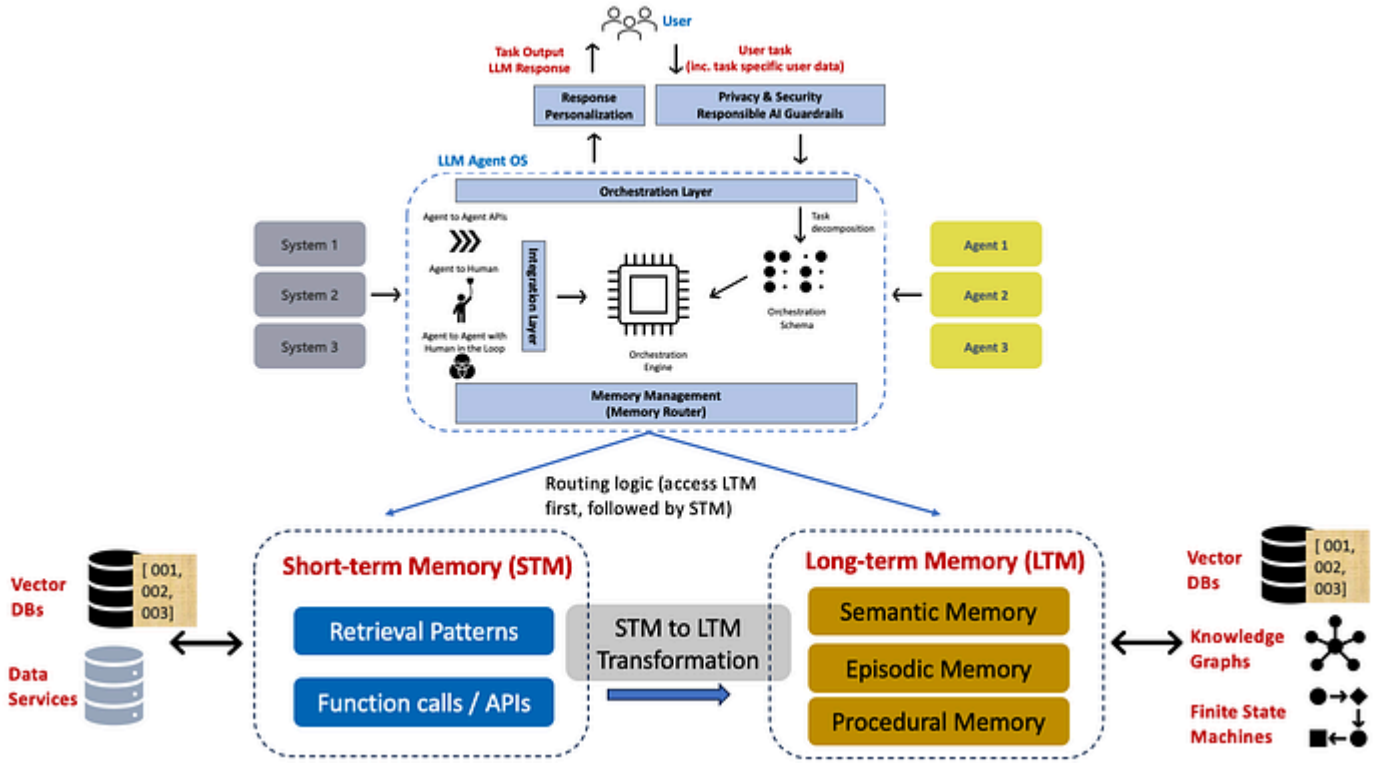


Fig. 6: Agentic AI Memory Management (Image by Author)

4. Agentic AI Risk Management

With more and more agentic AI systems getting deployed in production, we are seeing increasing focus on their **risks**. Rather than creating a new list, I tried to consolidate the risks identified in the below two references:

1. OWASP whitepaper: Agentic AI — Threats and Mitigations, 2025.
2. IBM whitepaper: Accountability and Risk Matter in Agentic AI, 2025.

R1–15 refer to the risks identified in [1]. The ones in brackets () refer to the corresponding risks identified in [2]. R16: Persona-driven **Bias**, e.g., is quite interesting, which has been identified in [2], but is missing from [1].

- R1: Misaligned & Deceptive Behaviors (Dynamic Deception)
- R2: Intent Breaking & Goal Manipulation (Goal Misalignment)
- R3: Tool Misuse (Tool/ API Misuse)
- R4: Memory Poisoning (Agent Persistence)
- R5: Cascading Hallucination Attacks (Cascading System Attacks)

(**Security** Vulnerabilities)

- R6: Privilege Compromise

- R8: Unexpected RCE & Code Attacks

(Operational **Resilience**)

- R9: Resource Overload
- R10: Repudiation & Untraceability

(Multi-agent **Collusion**)

- R11: Rogue Agents in Multi-agent Systems
- R12: Agent Communication Poisoning
- R13: Human Attacks on Multi-agent Systems

(Human **Oversight**)

- R14: Human Manipulation
- R15: Overwhelming Human in the Loop
- R16: (Persona-driven Bias)

The interesting part from a risk mitigation point of view is that their mitigation is often left to a central **guardrails** layer. However, this is not realistic and

> implemented in their respective platform components / layers — which has a direct impact on the overall solution architecture.

The agentic AI component risk-architecture mapping is depicted in Fig. 7.

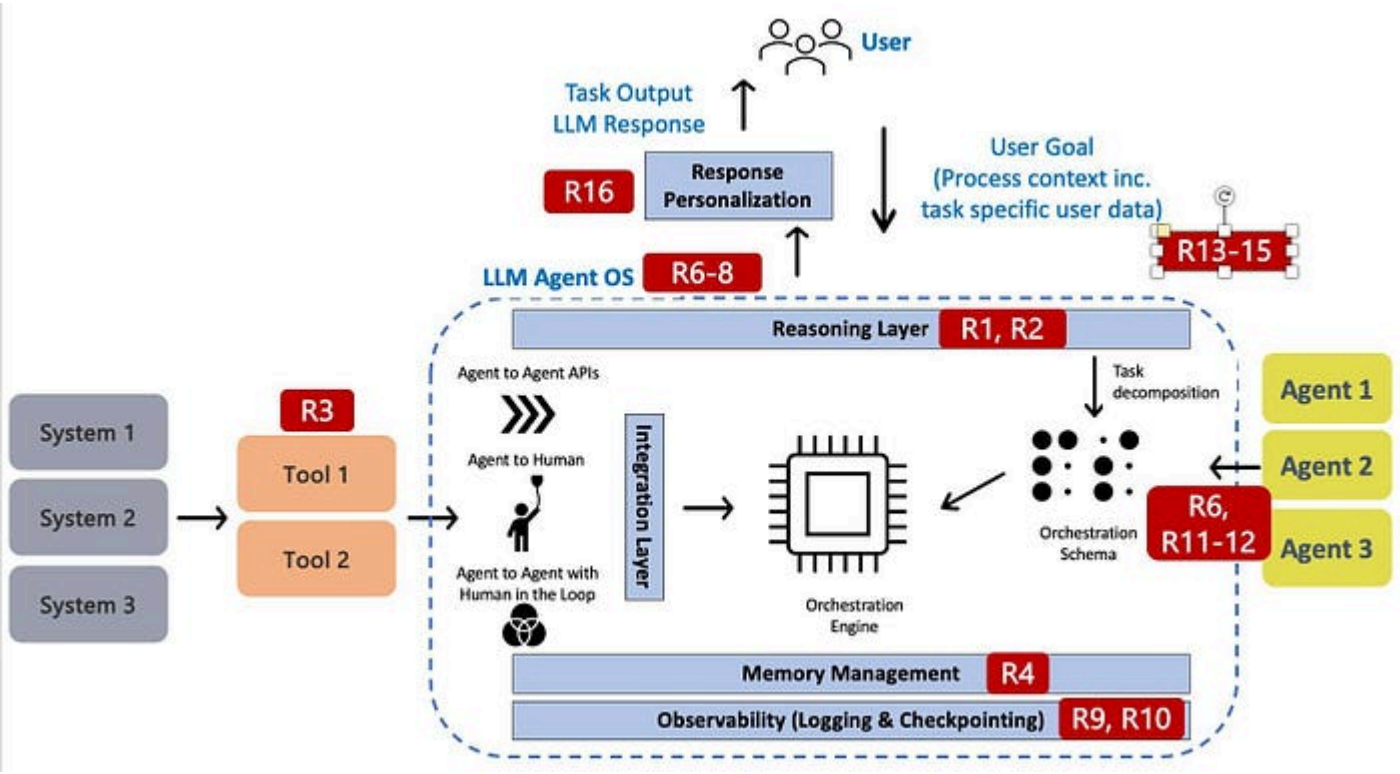


Fig. 7: Agentic AI risk mapping to platform architecture (Image by Author)

5. Human-in-the-Loop Intervention Strategy for AI Agents

that appropriate **UI/UX** needs to be designed to allow human intervention for the right task at the right time. (Similar to agentic state checkpointing, the frequency and format used to solicit human feedback matters.)

Below is a list of the key human intervention points to plan for — illustrated in Fig. 8:

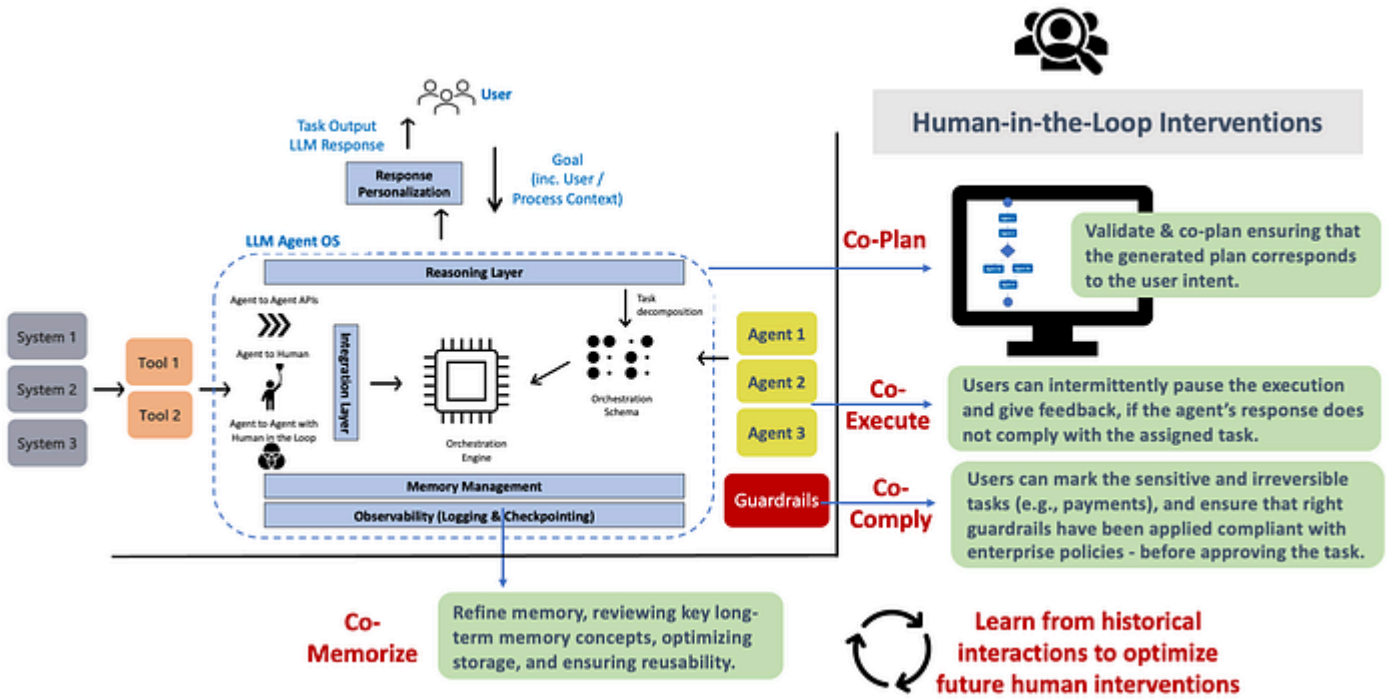


Fig. 8: Human-in-the-Loop intervention points of Agentic AI (Image by Author)

- **Co-execute:** Users can intermittently pause (suspend) the execution and give feedback, if the agent's / tool's response does not comply with the assigned task; or the human feels that the agent will not be able to achieve its (long-term) goal.
- **Co-comply:** Users can mark the critical and irreversible tasks (e.g., payments), and ensure that right guardrails have been applied compliant with enterprise policies — before approving the task.
- **Co-memorize:** Refine memory, reviewing key long-term memory concepts, optimizing storage, ensuring reusability and agent performance optimization.

This is complemented by a continuous improvement module that learns from historical interactions to optimize future human interventions.

6. Conclusion

Enterprise adoption of autonomous AI agents is currently going through a trust and reliability crisis. Towards this end, we outlined a framework to integrate humans as first-class citizens in the agentic lifecycle.

compliance perspective, humans provide the necessary oversight / guardrails for autonomous agents.

We believe that an effective human-in-the-loop (HITL) strategy has the potential to streamline multiple agentic aspects, e.g., planning, personalization and compliance (accountability); and as such will be critical in driving the enterprise adoption of agentic AI.

#agentic-ai #human-in-the-loop #agentic-ai-framework #llm-guardrails #risk-management